

BCC202 - Estruturas de Dados I

Aula 16: Ordenação: *Quick Sort*

Pedro Silva

Universidade Federal de Ouro Preto, UFOP
Departamento de Computação, DECOM
Email: silvap@ufop.edu.br



Conteúdo

Introdução

Execução

Implementação

Considerações Finais

Bibliografia

Introdução

Visão Geral

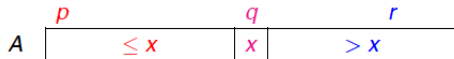
- ▶ Proposto por Hoare em 1960 e publicado em 1962.
- ▶ É o algoritmo de ordenação interna mais rápido que se conhece para **uma ampla variedade** de situações.
- ▶ O algoritmo *QuickSort* segue o paradigma de **divisão-e-conquista**.

Ordenar um subarranjo típico $A[p..r]$ (Cormen et al., 2011).

- ▶ **Divisão**: Particionar o arranjo $A[p..r]$
 - ▶ $A[p..q-1]$
 - ▶ $A[q+1..r]$
 - ▶ cada elemento de $A[p..q-1]$ é menor ou igual a $A[q]$
 - ▶ $A[q]$ é menor ou igual a cada elemento de $A[q+1..r]$
 - ▶ Calcular o índice q como parte desse procedimento de particionamento
- ▶ **Conquista**: Ordenar os dois subarranjos $A[p..q-1]$ e $A[q+1..r]$ por **chamadas recursivas ao *QuickSort***.
- ▶ **Combinação**: Como os subarranjos já estão ordenados, não é necessário nenhum trabalho para combiná-los: o arranjo $A[p..r]$ inteiro agora está ordenado.

Divisão e Conquista

O que podemos afirmar sobre o elemento que está na posição q , o **pivô**?



$$A[p \dots q - 1] \leq A[q] < A[q + 1 \dots r]$$

Partição em subproblemas

- ▶ A parte mais delicada do método é o processo de **partição**.
- ▶ O vetor $v[p \dots r]$ é rearranjado por meio da escolha arbitrária de um **pivô** x .
- ▶ O vetor v é particionado em duas partes:
 - ▶ Parte **esquerda**: *chaves* $\leq x$.
 - ▶ Parte **direita**: *chaves* $\geq x$.

Partição em subproblemas

► **Exemplo de algoritmo para o particionamento:**

1. Escolha arbitrariamente um pivô x .
2. Percorra o vetor a partir da esquerda até que $v[i] \geq x$.
3. Percorra o vetor a partir da direita até que $v[j] \leq x$.
4. Troque $v[i]$ com $v[j]$.
5. Continue este processo até os apontadores i e j se cruzarem.

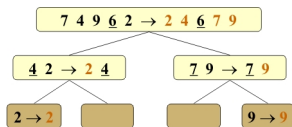
Partição em subproblemas

- ▶ Concluído o particionamento, o vetor $v[p \dots r]$ está particionado de tal forma que:
 - ▶ Os itens em $v[p]$, $v[p + 1]$, ..., $v[j]$ são menores ou iguais a x .
 - ▶ Os itens em $v[i]$, $v[i + 1]$, ..., $v[r]$ são maiores ou iguais a x .
- ▶ Pode-se concluir então que o pivô x encontra-se na posição correta de ordenação.

Execução

Árvore Binária

- ▶ A execução do **QuickSort** pode ser facilmente descrita por uma árvore binária:
 - ▶ Cada nó representa uma chamada recursiva do **QuickSort**.
 - ▶ O nó raiz é a chamada inicial.
 - ▶ Os nós folha são vetores de tamanho 0 ou 1 (casos base).



A seguir, mais detalhes sobre o passo a passo do *QuickSort*.



Animação



Vídeo

Exemplo do algoritmo de Partição

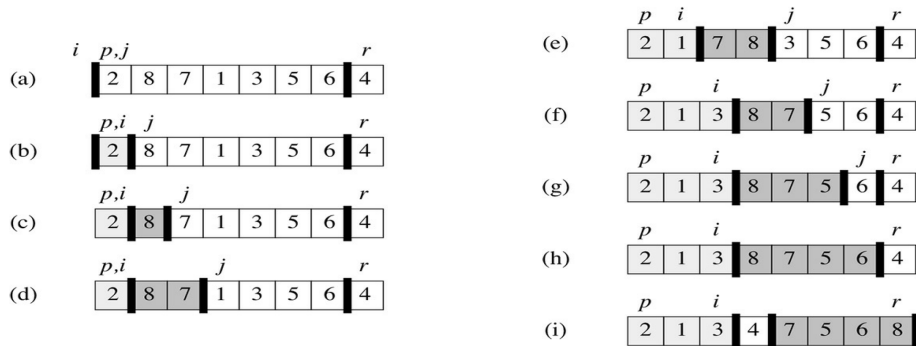


Figura: QuickSort: operação de *partição* (Cormen et al., 2011)

Implementação

Pseudocódigo: QuickSort

A seguir, o pseudocódigo do *QuickSort*.

1 **Algorithm:** QUICKSORT

Input: $\text{int}^* v, \text{int } p, \text{int } r$ // $p = \text{Left (esquerda)}$; $r = \text{Righty (direita)}$

2 **begin**

3 **if** $p < r$ **then**

4 $q \leftarrow \text{PARTITION}(v, p, r)$

5 $\text{QUICKSORT}(v, p, q - 1)$

6 $\text{QUICKSORT}(v, q + 1, r)$

7 **end**

8 **end**

Pseudocódigo: Partition Implementação I

A seguir, um possível pseudocódigo do procedimento de *partição*.

1 **Algorithm:** PARTITION

Input: int* v, int p, int r

Output: int // *índice q*

2 **begin**

3 $x \leftarrow v[r]$ // *x é o pivô*

4 $i \leftarrow p - 1$

5 **for** $j \leftarrow p$ **to** $j < r$ **do**

6 **if** $v[j] \leq x$ **then**

7 $i \leftarrow i + 1$

8 trocar $v[i]$ com $v[j]$

9 **end**

10 **end**

11 trocar $v[i + 1]$ com $v[r]$

 // *Pivô está na posição r que vai ser trocado com o elemento da posição i+1*

12 **return** $i + 1$ // *Complexidade: $O(n) = n \leftarrow r - p + 1$*

13 **end**

Pseudocódigo: Partition Implementação II

```
1  Algorithm: PARTITION
   Input: int* v, int p, int r
   Output: int // índice q
2  begin
3       $x \leftarrow v[(p+r)/2]$  // x é o pivô
4       $i \leftarrow p; j \leftarrow r$ 
5      while  $i \leq j$  do
6          while  $v[i] < x$  and  $i < r$  do
7               $i \leftarrow i + 1$ 
8          end
9          while  $v[j] > x$  and  $j > p$  do
10              $j \leftarrow j - 1$ 
11         end
12         if  $i \leq j$  then
13             trocar  $v[i]$  com  $v[j]$ 
14              $i \leftarrow i + 1; j \leftarrow j - 1$ 
15         end
16     end
17     // Pivô está na posição i + 1
18     return  $i + 1$  // Complexidade:  $O(n) = n \leftarrow r - p + 1$ 
19 end
```

Tempo, Comparação e Espaço

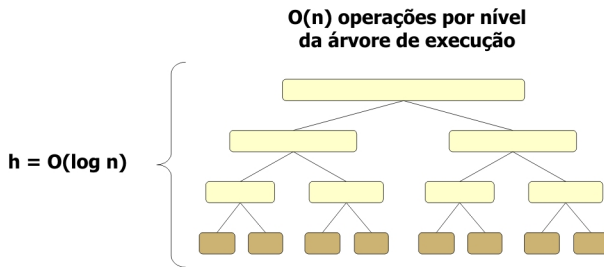
QuickSort é um algoritmo não estável.

- ▶ Complexidade de tempo no pior caso
 - ▶ $O(n^2)$ comparações
 - ▶ $O(n)$ trocas
- ▶ Complexidade de tempo no melhor caso
 - ▶ $O(n \log n)$ comparações
 - ▶ 0 trocas
 - ▶ $O(n \log n) + 0 = O(n \log n)$
- ▶ Complexidade de espaço/consumo de espaço
 - ▶ Extra: $O(n)$

Melhor Caso

$$C(n) = 2 * C(n/2) + n = n \log n = O(n \log n)$$

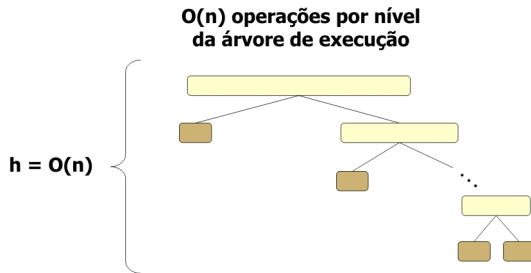
- ▶ Ocorre quando o problema é sempre dividido em subproblemas de igual tamanho após a partição.



Pior Caso

$$C(n) = O(n^2)$$

- Ocorre quando, sistematicamente, o **pivô** é escolhido como sendo um dos extremos de um arquivo já ordenado.



Pior Caso

- ▶ O pior caso pode ser evitado empregando pequenas modificações no algoritmo. Algumas opções:
 - ▶ Escolher o pivô aleatoriamente.
 - ▶ Escolher três itens quaisquer do vetor e usar a mediana dos três como pivô.
 - ▶ **"Embaralhar"** o vetor original antes de iniciar a ordenação. Um bom algoritmo é o de **Fischer-Yates** ($O(n)$):

```
1 n = tamanhoDoVetor
2 para cada i entre n e 2
3     sorteie j como um número entre 1 e i
4     se i e j forem diferentes, troque os elementos i e j entre si
```

Caso Médio

$$C(n) \approx 1.386n \log n - 0,846n = O(n \log n)$$

- ▶ Análise por **Sedgewick e Flajolet (1996, p. 17)**.
- ▶ A proporção das divisões não será sempre constante.
- ▶ Ocorre quando há uma mistura de divisões boas e ruins.
- ▶ Perceba que o caso médio está muito mais próximo do melhor caso do que do pior caso.

QuickSort RECURSIVO: Características

Vantagens

- ▶ É extremamente eficiente para ordenar arquivos.
- ▶ Requer apenas uma pequena pilha como memória auxiliar.
- ▶ Requer $O(n \log n)$ comparações em média (caso médio) para ordenar n itens.

Desvantagens

- ▶ Tem um pior caso com $O(n^2)$ comparações.
- ▶ Implementação delicada e difícil: um pequeno engano pode levar a efeitos inesperados.
- ▶ O método **não é estável**.

1 Algorithm: QUICKSORT-ITERATIVO**Input:** $\text{int}^* v, \text{int } p, \text{int } r$ **2 begin****3** alocar $\text{pilha_p}[0 \dots (r - p + 1)]$ e $\text{pilha_r}[0 \dots ((r - p + 1))]$ **4** $\text{pilha_p}[0] \leftarrow p$ $\text{pilha_r}[0] \leftarrow r$ $t \leftarrow 0$ **5** **while** $t \geq 0$ **do****6** $p \leftarrow \text{pilha_p}[t]$ **7** $r \leftarrow \text{pilha_r}[t]$ **8** $-- t$ **9** **if** $p < r$ **then****10** $j \leftarrow \text{PARTITION}(v, p, r)$ **11** $++ t$ **12** $\text{pilha_p}[t] \leftarrow p$ **13** $\text{pilha_r}[t] \leftarrow j - 1$ **14** $++ t$ **15** $\text{pilha_p}[t] \leftarrow j + 1$ **16** $\text{pilha_r}[t] \leftarrow r$ **17** **end****18** **end****19** desalocar $\text{pilha_p}[0 \dots (r - p + 1)]$ e $\text{pilha_r}[0 \dots ((r - p + 1))]$ **20 end**

Pilha de Recursão v.s. Pilha Explícita

- ▶ O que é colocado em cada uma das pilhas?
- ▶ Qual intervalo do vetor é empilhado em cada caso?

Análise *QuickSort* iterativo

Vantagens

- ▶ É extremamente eficiente para ordenar arquivos de dados.
- ▶ Necessita de apenas uma pequena pilha como memória auxiliar.

Desvantagens

- ▶ Sua implementação é delicada e difícil.

Melhorias no *QuickSort*

- ▶ O pivô é a mediana de três:
 - ▶ Não empilhar quando tem apenas um item.
- ▶ Melhor ainda:
 - ▶ Usar algoritmo de inserção (*InsertionSort*) para vetores pequenos (menos de 10 ou 20 elementos).
- ▶ Escolha correta do lado a ser empilhado.

- ▶ RESULTADO:

Melhoria no tempo de execução de 25% a 30%

Considerações Finais

- ▶ **QuickSort**: divisão e conquista.
- ▶ Duas implementações: **Recursiva** e **Iterativa**.
- ▶ Sobre a complexidade do QuickSort.

Quadro Comparativo dos métodos de ordenação

Algoritmo	Comparações			Movimentações			Espaço	Estável	In situ
	Melhor	Médio	Pior	Melhor	Médio	Pior			
Bubble	$O(n^2)$			$O(n^2)$			$O(1)$	Sim	Sim
Selection	$O(n^2)$			$O(n)$			$O(1)$	Não*	Sim
Insertion	$O(n)$	$O(n^2)$		$O(1)$	$O(n^2)$		$O(1)$	Sim	Sim
Merge	$O(n \log n)$			$O(n \log n)$			$O(n)$	Sim	Não
Quick	$O(n \log n)$		$O(n^2)$	–			$O(n)$	Não*	Sim

* Existem versões estáveis.

Na próxima aula

ShellSort.

Exercício 01

- ▶ Dada a sequência de números: 3 4 9 2 5 1 8.
- ▶ Ordene em ordem crescente utilizando o algoritmo aprendido em sala (**QuickSort**), apresentando a sequência dos números a cada passo (Teste de Mesa).

Bibliografia

Bibliografia

Os conteúdos deste material, incluindo figuras, textos e códigos, foram extraídos ou adaptados de:

- ▶ **Slides MO417 - Complexidade de Algoritmos I**, elaborados por Cid Carvalho de Souza, Cândida Nunes da Silva e Orlando Lee e revisado por Zanoni Dias em agosto de 2011, <https://www.ic.unicamp.br/~zanoni/teaching/mo417/2011-2s/aulas/handout/04-ordenacao.pdf>. Acessado em 2021.

 Cormen, Thomas H. and Leiserson, Charles E. and Rivest, Ronald L. and Stein, Clifford.

Introduction to Algorithms.

The MIT Press, 2011.